Project 3: Link analysis

Algorithms for massive datasets - a.a.25/26

Michele Ghirardelli

September 10, 2026

Contents

1 Introduction

This project deals with the creation of a ranking system based on the PageRank, an algorithm made famous by Google, to evaluate how applicable and how scalable it is in different domains. The project is composed by a notebook file with the implementation of the page rank and the report, all the material can be found in the repository `https://github.com/ghirailghiro/AMD-Ghirardelli-Michele`

2 Data

In this section we describe which dataset was used for the project and which subset of it was employed.

2.1 Dataset

For the dataset we used the "New York Times Articles & Comments (2020)" collection published on Kaggle under the CC BY-NC-SA 4.0 license, which contains all comments and articles from January 1, 2020 to December 31, 2020. The dataset is in CSV format and contains approximately 5M comments with 23 features and 16k articles with 11 features. The dataset was last accessed on 09/09/2026.

2.1.1 Subset utilized

For the construction of the graph we selected the comments dataset contained in the file "nyt-comments-2020.csv". In addition, a flag was introduced in order to select a subset of the months of comments. In this dataset each row is a unique comment identified by a commentID, and every comment is associated with the user who wrote it. Moreover, a comment may be a reply to another one: in that case its parentID column is filled with the commentID of the parent comment. The dataset present some particular cases:

- **Self-loops**: users replying to themselves
- **Anonymous users**: user that didn't have an account

When the `FULL_DATASET` flag is disabled, only one month (2020-01) is selected, for performance reasons. When the `EDGES_RANGES_ANALYSIS` flag is enabled, the portion of the dataset selected by `FULL_DATASET` is further split by taking the first k edges only. This second flag is used exclusively in the scalability section, i.e. for performance measurements, and not for the actual PageRank computation.

3 Graph definition

In this section we define the graph on which PageRank is computed. The nodes are the users who posted comments, and we define an edge from user u to user v if u replied to a comment written by v. Therefore $u \to v$ represents an act of attention towards v, analogous to the hyperlink in the PageRank of web pages. Since the nodes are users, PageRank can express how important or authoritative a user is with respect to the others. Consequently, the dangling nodes, i.e. the dead ends, are those users who never replied to anyone and only wrote root comments.

3.1 Pre-processing

From the dataset we need to extract the pairs (source user, destination user) starting from the comments data:

- **Edge definition**: since `parentID` refers to a comment and not to a user, a join on the same dataset was required in order to retrieve the author of the parent comment.
- **Self-loops**: users replying to themselves were removed, since in PageRank they act as small spider traps that inflate the score.
- **Deduplication**: if u replies to v seven times, possibly under different articles, only a single edge is kept. This is consistent with the representation in which the value is constant along each column.
- **Anonymous users**: anonymous users were filtered out, since several of them sharing the same identifier would collapse into a single node with an artificially high in-degree.

Step	**Value**	**Percentage**
Total comments	4,986,461	—
Comments with parent (replies)	2,160,907	43.34%
Orphans discarded	14,082	0.65%
Self-loops removed	10,363	0.52%
Anonymous edges removed	0	0.00%

Table 1: Pre-processing pipeline: from raw comments to the final graph. Percentages refer to the set each step is applied to: replies are a fraction of all comments, orphans a fraction of the replies, and self-loops a fraction of the distinct (source, destination) pairs.

4 Algorithms

4.1 PageRanking

PageRank is based on the idea of the random surfer: at each step the surfer follows one of the links of the current page at random, simulating where a random user could end up. In our case we want to simulate a reader who jumps from user to user by following the replies.

$$v' = \beta \left(Mv + \frac{S}{n} \right) + \frac{1-\beta}{n} \tag{1}$$

The transition matrix M is defined so that each non-zero entry of column j equals $1/d_j$, where d_j is the out-degree of user j, i.e. the number of distinct users to whom j replied; each column therefore sums to one. However, since dangling nodes are present — users who only wrote root comments and never replied — the matrix is not column-stochastic. For this reason the term S/n is added to the result of the matrix-vector product, where S is the total PageRank mass currently held by the dangling nodes. Finally, the taxation parameter $\beta = 0.85$ is used, a value typically adopted in the literature and suggested in [2]. The residual, i.e. the difference between the new and the previous rank vector, is therefore expected to decay as β^t.

4.2 Implementations

The matrix is stored in a sparse representation for reasons of space efficiency. Two implementations were therefore developed, following what is presented in the reference book: a sequential implementation operating on the sparse matrix, and the same implementation expressed in the MapReduce paradigm. The low-level implementation relies on three arrays: one containing the source nodes, one containing the destination nodes, and a third one holding, for each user, the number of outgoing edges, i.e. its out-degree.

The loop then iterates over every source node and, for each of them, its contribution to the rank vector is computed and scattered over its outgoing edges.

5 MapReduce Implementation

Three main data structures are initialised:

- `v_spark`: the rank vector, initialised through `parallelize` as an RDD of pairs (node, rank), covering all the n nodes of the graph.
- `sparse_matrix_spark`: the transition matrix, stored as an RDD of pairs (source, list of destinations).
- `zeros_for_missing_nodes`: an RDD of pairs (node, 0.0) over all the n nodes.

All these structures are partitioned by k, initialised with `sc.defaultParallelism`.

The implementation consists of a loop of 50 iterations. The dangling set is initialised beforehand, so that the value S can be computed at each step. The transition matrix is then joined with the rank vector, and a `flatMap` is applied to the result of this join in order to compute the contribution of each node j. This produces a list of key-value pairs, where the keys are the identifiers of the destination nodes and the values are the rank of the source divided by its out-degree. A `reduceByKey` with a sum is then applied, followed by a union with a vector of zeros, since the `flatMap` step may have reduced the total length of the vector; finally, a `mapValues` applies the taxation and the dangling contribution to every entry.

The new vector is then joined with the previous one, and a map followed by a reduce with a sum is used to compute the residual, before starting the next iteration.

6 Scalability

We implemented both a dense and a sparse version of the algorithm in NumPy, in order to draw some considerations about the scalability of the two solutions and to show that the second one is preferable. In addition, the sparse version was also implemented in the MapReduce paradigm, which turns out to be considerably slower. This is certainly due to the fact that the experiments were executed on Google Colab, where the available resources are limited: the overhead required to build and manage the distributed data structures leads to a considerably longer execution time.

Nodes	**Edges**	**Time per iteration (s)**		**Memory (MB)**	
		Dense	Sparse	Dense	Sparse
500	28	0.000516	0.000384	1.91	0.0004
1,000	62	0.005133	0.000203	7.63	0.0009
2,000	324	0.002724	0.000843	30.52	0.0049
4,000	1,288	0.002962	0.002687	122.07	0.0197
262,184	1,967,288	—	3.05	**∼512,000***	30.018

Table 2: Dense versus sparse representation on induced subgraphs of increasing size. The last row reports the full graph: the dense matrix would require about 550 GB and cannot be allocated, while the sparse representation runs in a few minutes. On the same subgraph the two implementations produce the same rank vector, with a maximum absolute difference below 10^{-18}. *** This value is projected following the increasing memory footprint, storing the dense matrix would require at least $n^2 \times 8$ bytes, i.e. about 512 GiB**

Here we can see the differences between the spark and numpy implementation:

Implementation	Nodes	Tot time (min)	Time/iter (s)	Mem (MB)
NumPy	262,184	2.54	3.05	30.0
MapReduce	262,184	9.95	11.94	—

Table 3: Execution time of the two implementations on the full graph, over 50 iterations. Memory is not reported for Spark, since RDDs are partitioned and materialised lazily, so their footprint depends on the execution plan and is not directly comparable with the allocation of a NumPy array.

7 Data Analysis

In this section we describe the structure of the graph, and in particular the fraction of nodes having both incoming and outgoing edges. These are the nodes through which the rank can actually flow during the exploration of the random surfer: a node with no outgoing edges can only accumulate rank, while a node with no incoming edges can only give it away.

Step	Value	% of graph nodes
Graph nodes	262,184	100.00
Users who replied (out-degree > 0)	196,093	74.79
Users who received replies (in-degree > 0)	181,962	69.40
Users in both sets	115,871	44.19
Final edges	1,967,288	—
Average out-degree	7.50	—

Table 4: Structure of the reply graph. The dataset contains 403,025 distinct users, but only 262,184 of them (65.05%) appear in the graph: the remaining ones only wrote root comments and never received any reply, so they have neither incoming nor outgoing edges.

7.1 Experiments

The experiments were carried out on both the NumPy and the MapReduce implementations, and the result is identical for the two of them. The run performed on the whole graph, consisting of 262,184 nodes and 1,967,288 edges, was executed for 50 iterations, reaching a residual of 6.18×10^{-7}.

8 Results

Table 5 reports the top-20 users by PageRank. As can be seen, the ratio between the number of replies received and the number of comments posted varies considerably among them. This shows that PageRank is not merely a measure

of who received the largest number of replies: what matters is also who those replies come from, since a reply coming from a user who is himself authoritative carries more weight than one coming from an occasional commenter.

#	User	PageRank	In-degree	Comments	Ratio (%)
1	Socrates	0.001724	3549	4279	82.9
2	KMW	0.001146	2967	2190	135.5
3	**Marge Keller**	0.001064	2547	7049	**36.1**
4	Red Sox, '04, '07, '13, '18	0.000946	2153	1636	131.6
5	ChristineMcM	0.000901	2146	2328	92.2
6	AACNY	0.000832	2206	4891	45.1
7	Roland Deschain	0.000804	1841	1310	140.5
8	Bruce Rozenblit	0.000782	1813	959	189.1
9	NM	0.000772	1801	2197	82.0
10	Mon Ray	0.000748	1774	1702	104.2
11	**Nicholas Kristof**	0.000745	1470	222	**662.2**
12	Phyliss Dalmatian	0.000700	1580	3803	41.5
13	michjas	0.000654	1720	1816	94.7
14	Ronald B. Duke	0.000639	1736	1072	161.9
15	Jonathan Katz	0.000618	1550	1956	79.2
16	Innisfree	0.000607	1354	1435	94.4
17	John	0.000597	1575	1678	93.9
18	Deirdre	0.000531	1413	2619	54.0
19	Jay Orchard	0.000526	1328	1947	68.2
20	David H	0.000520	1357	1946	69.7

Table 5: Top-20 users by PageRank, compared with their in-degree, number of posted comments, and the ratio between in-degree and number of comments.

These claims can be verified by observing that:

- Kristof ranks 11th with 1,470 incoming replies, ahead of michjas (1,720), Ronald B. Duke (1,736) and John (1,575);
- AACNY, with 2,206 incoming replies, ranks 6th, behind ChristineMcM who has 2,146;
- Phyliss Dalmatian, with 1,580 incoming replies, ranks 12th, ahead of users with a higher in-degree.

Moreover, we can state that PageRank does not blindly follow the volume of posted comments either, as the following examples show:

- Marge Keller, by far the most prolific user with 7,049 comments, only ranks 3rd;
- AACNY, with 4,891 comments, ranks 6th;
- Bruce Rozenblit, with 959 comments (one seventh of Marge Keller), ranks 7th.

The last interesting observation concerns the outlier Nicholas Kristof, who is a journalist of the New York Times. This gives us a meaningful piece of information: this PageRank identifies as important those authoritative people who comment under the articles.

The following table reports the cross-validation of the results between the MapReduce and the NumPy implementations, showing that the two of them produce the same converged output.

Check	**Result**
Users in the top-20 of both implementations	20 / 20
Users appearing in the same position	20 / 20
Maximum difference in PageRank value	3.9×10^{-18}

Table 6: Cross-validation between the two implementations. Both produce exactly the same ranking, in the same order, with a maximum absolute difference at machine precision

In order to verify the recursive nature of the algorithm, we compared the average PageRank of the users who reply to two members of the top-20 — Nicholas Kristof, ranked 11th, and Marge Keller, ranked 3rd — with the average PageRank of the whole graph, which is $1/n = 3.81 \times 10^{-6}$. The users replying to Kristof have an average PageRank of 3.72×10^{-5}, about 9.8 times the global average, and six of them belong to the top-20; the users replying to Keller have an average PageRank of 4.45×10^{-5}, about 11.7 times the global average, with twelve of them in the top-20.

In both cases the users providing the incoming edges are roughly one order of magnitude above the average of the graph: the rank does not come from arbitrary users, but from nodes that are themselves well positioned, which is exactly the behaviour we expect from the algorithm. It is worth noting that Keller receives replies from slightly more authoritative users than Kristof does, and indeed she ranks higher; his position is therefore not explained by a better quality of his interlocutors.

These two outliers show that PageRank is based neither on the volume of activity alone — how many comments a user posts — nor on the in-degree alone. What distinguishes Kristof is his efficiency: with only 222 comments, one order of magnitude fewer than any other user in the top-20, he still reaches the eleventh position. His ratio between replies received and comments posted is 662%, against 36% for Keller and 189% for the second best value in the table. No ranking based on the volume of activity or on the raw in-degree would have identified him.

9 Conclusions

From the results obtained we can conclude that the rank is able to flow through the graph and to provide meaningful information about the dataset, and that

the final ranking is not simply a matter of how many replies a user received.

This approach may however underestimate the actual importance of a user, since it does not take into account the possible @-mentions contained in the body of the comments. Many replies begin with an explicit mention of Nicholas Kristof, but are formally attached to a different comment through their `parentID`: the edge is therefore directed towards another user, and the attention actually addressed to him is not captured by the graph.

10 Declaration

"I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content."

References

[1] Benjamin Dornel, *New York Times Articles & Comments (2020)*. Available at `https://www.kaggle.com/datasets/benjaminawd/new-york-times-articles-comments-2020/data`. Accessed: 2026-09-09.

[2] Leskovec, Rajaraman, Ullman, *Mining of Massive Datasets*, 3rd ed., Cambridge University Press, 2020.